

Projeto Meninas

Documento de explicacao funcional e tecnica do backend Spring Boot do projeto `meninas`. O foco aqui e explicar como o sistema esta organizado, como os dados circulam e qual e o papel de cada classe principal.

1. Visao Geral

Este projeto e um backend em Java com Spring Boot, conectado a PostgreSQL, que expoe endpoints HTTP para autenticar usuarios, cadastrar usuarios, publicar tweets e cadastrar atividades com foto. Ele usa JWT para autenticar chamadas protegidas.

Tecnologias

- Spring Boot 3.4.1
- Spring Web
- Spring Security
- OAuth2 Resource Server
- Spring Data JPA
- PostgreSQL
- Lombok

Objetivo do backend

- Autenticar usuarios por token
- Controlar acesso por role
- Persistir usuarios, roles, tweets, atividades e fotos
- Expor API para frontend ou Postman

2. Estrutura do Projeto

O codigo esta dividido em pacotes com papeis bem definidos:

- `Config` : configuracoes de seguranca, token e bootstrap do admin.
- `Controller` : endpoints REST expostos pela aplicacao.
- `Controller.dto` : objetos de entrada e saida da API.
- `Entity` : mapeamentos JPA das tabelas do banco.
- `Repository` : acesso aos dados com Spring Data JPA.
- `Service` : regras de negocio mais concentradas.
- `Util` : utilitarios auxiliares, como geracao de chave.

3. Dependencias Principais

O arquivo `pom.xml` define as bibliotecas do projeto. As principais sao:

- `spring-boot-starter-web` : cria a API HTTP.

- `spring-boot-starter-security` : aplica autenticao e autorizacao.
- `spring-boot-starter-oauth2-resource-server` : valida JWT nas rotas protegidas.
- `spring-boot-starter-data-jpa` : mapeia objetos Java para tabelas SQL.
- `postgresql` : driver de conexao com o banco.
- `lombok` : reduz codigo repetitivo em getters, setters e construtores.

4. Classe Inicial

`MeninasApplication`

E a porta de entrada da aplicacao. Quando o programa sobe, o Spring Boot inicia o contexto, monta todos os beans, conecta no banco e registra as rotas HTTP.

5. Configuracao e Seguranca

`SecurityConfig`

Esta classe define as regras de seguranca do sistema.

- Libera sem login os endpoints `POST /login` e `POST /token`.
- Exige autenticao nas outras rotas.
- Desliga CSRF porque a API esta stateless.
- Configura o backend para validar JWT em todas as chamadas protegidas.
- Cria os beans `JwtEncoder`, `JwtDecoder` e `BCryptPasswordEncoder`.

Fluxo de seguranca: o usuario pede um token em `/token`, recebe um JWT e usa esse JWT como `Bearer Token` para acessar rotas protegidas como `/tweets`.

`TokenConfig`

Guarda parametros do token, como emissor e tempo de expiracao. Hoje ele fornece:

- `jwt.issuer` : emissor do token, padrao `meninas-api`.
- `jwt.expires-in` : tempo de vida em segundos, padrao `3600`.

`AdminUserConfig`

Roda na inicializacao da aplicacao. O objetivo e garantir que existam as roles basicas e que o usuario administrador exista ou seja atualizado.

- Cria `ROLE_ADMIN` e `ROLE_BASIC` se faltarem.
- Procura o usuario admin pelo username configurado.
- Atualiza a senha do admin e garante que ele tenha role de administrador.

Observacao: para ambiente de producao, a senha do admin nao deveria ficar no `application.properties`. O ideal seria variavel de ambiente.

6. Entidades do Banco

Usuario

Representa um usuario do sistema.

- `usuarioId` : identificador UUID.
- `username` : nome unico do usuario.
- `password` : senha criptografada com BCrypt.
- `roles` : conjunto de permissoes do usuario.

Tambem possui o metodo `isLoginCorrect` , que compara a senha enviada no login com a senha criptografada no banco.

Role

Representa os perfis de acesso. Hoje existem dois valores definidos:

- `ROLE_ADMIN` : administrador.
- `ROLE_BASIC` : usuario comum.

Tweet

Representa uma postagem curta feita por um usuario autenticado.

- `tweetId` : id gerado por sequencia.
- `usuario` : dono do tweet.
- `content` : texto de ate 280 caracteres.
- `creationTimestamp` : data de criacao gerada automaticamente.

Atividade

Representa uma atividade que sera exibida no sistema. Ela funciona como um conteudo mais rico que o tweet.

- `titulo` : nome da atividade.
- `descricao` : descricao detalhada.
- `data` : data textual da atividade.
- `fotoNomeArquivo` : nome original do arquivo.
- `fotoContentType` : tipo MIME da imagem.
- `fotoDados` : bytes binarios da imagem.

O campo `fotoDados` esta com `@JsonIgnore` , para a listagem de atividades nao devolver o binario inteiro em JSON.

Foto

Existe uma entidade separada para foto, ligada a atividade, mas no estado atual do projeto a logica principal de foto em atividade ficou centralizada na propria entidade `Atividade` . A entidade `Foto` ainda existe, mas esta subutilizada.

7. Repositories

Os repositories fazem o acesso ao banco sem precisar escrever SQL manual na maior parte do tempo.

- `UsuarioRepository` : busca, salva e remove usuarios.
- `RoleRepository` : busca roles por nome.

- `TweetRepository` : salva e consulta tweets.
- `AtividadeRepository` : salva e consulta atividades.
- `FotoRepository` : salva e consulta fotos.

8. Services

`UsuarioService`

Concentra logica de usuario: salvar, buscar, atualizar, deletar e autenticar. Ele tambem possui um gerador de token proprio. Hoje existe uma sobreposicao entre `UsuarioService` e `TokenController` , porque os dois lidam com login/JWT.

Observacao: arquiteturalmente, o projeto ficaria mais limpo se a geracao de token estivesse em um unico lugar.

`AtividadeService`

Faz o CRUD basico de atividade: salvar, listar, buscar por id e deletar.

`FotoService`

Faz o CRUD basico de foto, mas atualmente o fluxo de atividade com foto foi simplificado para salvar a imagem diretamente em `Atividade` .

9. Controllers e Rotas

`TokenController`

Responsavel pelo endpoint `POST /token` .

- Recebe `username` e `password` .
- Busca o usuario no banco.
- Valida a senha com BCrypt.
- Monta o claim `scope` a partir das roles do usuario.
- Gera e devolve o JWT.

Exemplo de uso: `POST /token` com JSON de login. A resposta devolve `accesstoken` e `expiresIn` .

`UsuarioController`

- `POST /usuario` : cria usuario novo com role basica.
- `GET /usuario` : lista usuarios, mas exige `SCOPE_ADMIN` .

`TweetController`

- `POST /tweets` : cria tweet para o usuario autenticado.

O controller extrai o id do usuario a partir do token JWT, busca o usuario no banco e salva o tweet ligado a ele.

`AtividadeController`

- `POST /atividades` : cria atividade com titulo, descricao, data e foto opcional via `form-data` .
- `GET /atividades` : lista atividades.

- `GET /atividades/{id}/foto` : devolve a imagem da atividade.
- `DELETE /atividades/{id}` : remove atividade.

Esta parte ficou conceitualmente parecida com tweet, mas com mais campos e suporte a imagem.

AuthController

Existe um controller adicional para `/login` com validacao simplificada baseada em usuario fixo. Ele convive com `/token` , mas no estado atual o endpoint mais coerente com o restante do projeto e `/token` .

FotoController , CronogramaController e ExposicaoController

Sao endpoints auxiliares. `ExposicaoController` reutiliza as atividades como conteudo de exibicao. `FotoController` continua existindo para a entidade `Foto` .

10. Fluxos Principais

Fluxo de Login

1. Cliente envia `POST /token` com usuario e senha.
2. O backend valida a senha no banco.
3. O backend gera um JWT com `sub` , `scope` , `iss` e `expiracao`.
4. O cliente usa esse JWT como `Bearer Token` .

Fluxo de Tweet

1. Cliente faz login e recebe token.
2. Cliente envia `POST /tweets` com `content` .
3. O backend identifica o usuario pelo token.
4. O tweet e salvo ligado ao usuario.

Fluxo de Atividade com Foto

1. Cliente envia `POST /atividades` como `form-data` .
2. O backend recebe titulo, descricao, data e arquivo de foto.
3. A atividade e salva no banco com metadados da foto e bytes da imagem.
4. O cliente pode listar tudo em `GET /atividades` .
5. O cliente pode carregar a imagem em `GET /atividades/{id}/foto` .

11. application.properties

As configuracoes principais atuais sao:

- `server.port=8080`
- conexao PostgreSQL para banco `Meninas`
- chaves RSA para JWT em `classpath:chave_privada.pem` e `classpath:chave_publica.pem`
- bootstrap do admin com `admin.username` e `admin.password`

12. Pontos Fortes do Projeto

- JWT funcionando para autenticacao.
- Integracao com PostgreSQL funcional.

- Persistencia de tweet e atividade funcionando.
- Protecao de endpoints com role de administrador.
- Estrutura em camadas ja existe e esta compreensivel.

13. Pontos a Melhorar

- Consolidar login em um unico fluxo, preferencialmente `/token`.
- Tirar a senha do admin do arquivo e mover para variavel de ambiente.
- Decidir entre usar a entidade `Foto` ou manter a foto dentro de `Atividade`, para evitar duplicidade conceitual.
- Padronizar encoding dos arquivos para UTF-8 limpo.
- Adicionar testes automatizados de login, tweet e atividade.
- Criar endpoints de listagem e detalhe para tweets.

14. Resumo Final

O projeto Meninas e um backend Java com Spring Boot voltado para autenticacao com JWT e persistencia de conteudo. O modelo atual permite:

- criar usuarios e roles
- autenticar por token
- publicar tweets
- cadastrar atividades com imagem
- consultar o conteudo salvo no PostgreSQL

Em termos de arquitetura, ele ja tem as pecas fundamentais de uma API moderna: configuracao, controller, service, repository, entidade, seguranca e integracao com banco.

Documento gerado a partir do estado atual do workspace em 11/04/2026.